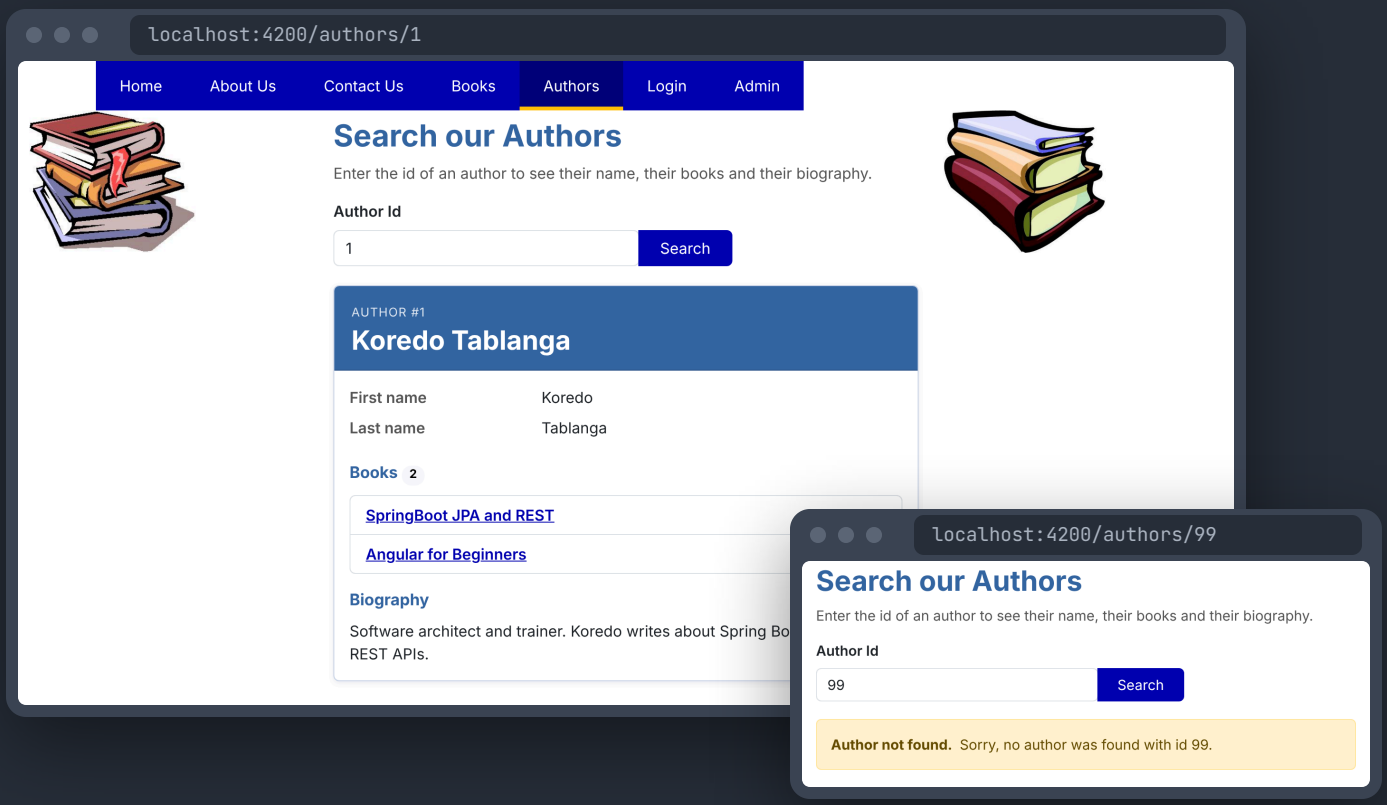
SEG3502 · ARCHITECTURE ET CONCEPTION LOGICIELLE

Laboratoire 7
Services Web de données

Option « Authors » du client Angular : on entre l'id d'un auteur, l'application interroge l'API REST Spring Boot et affiche l'auteur, ou un message s'il n'existe pas



ÉQUIPE

Brayan Adou — 300433616
Interface : menu Authors, HTML et CSS, tests d'affichage, README et rapport

Hjiyej Andaloussi Elghali — 300379897
Logique : service HTTP, routes, composants, tests et script de données

COURS
SEG3502 A00
Université d'Ottawa

REMISE
4 novembre 2026

DÉPÔT GITHUB
github.com/ralihji/Lab-7

01 Description du travail réalisé

L'exercice évalué du Lab 7 demande d'ajouter une option de menu « Authors » à la barre de navigation. Un clic affiche dans la zone Contenu un formulaire pour entrer l'id d'un auteur. Une fois le formulaire soumis, l'application fait une requête et affiche les informations de l'auteur s'il est trouvé, sinon un message approprié.

Le client Angular du Book Store interroge l'API REST du serveur Spring Boot (Kotlin, JPA, HATEOAS, MySQL). **Le serveur du code de départ n'a pas été modifié** : la route `GET /books-api/authors/{id}` existait déjà. Elle renvoie l'auteur au format HAL (`hal+json`) : ses données, le titre de ses livres et des liens hypermédias, dont un lien vers sa biographie.

- Le champ **Author Id** est vérifié avant l'envoi : il faut un entier supérieur à 0 (« 007 » devient 7). Sinon, un message apparaît sous le champ et aucune requête n'est envoyée.
- La recherche ouvre `/authors/:id` , sur le modèle de `/books/:id` pour les livres. L'adresse d'un auteur peut donc être ouverte directement, et les boutons précédent et suivant du navigateur fonctionnent.
- **Auteur trouvé** : prénom, nom, livres (chaque titre ouvre la page du livre) et biographie, obtenue en suivant le lien HATEOAS `bio` de la réponse.
- **Auteur introuvable** (réponse 404) : « Author not found. Sorry, no author was found with id 99. » D'autres messages couvrent un id invalide, un serveur arrêté et une erreur du serveur.

Circulation des données

1. AuthorsComponent (/authors)
Formulaire réactif. `onSubmit()` vérifie l'id puis ouvre `/authors/:id` .

→

2. AuthorComponent (/authors/:id)
Lit l'id dans l'adresse (`paramMap`), appelle `getAuthor(id)` puis `getAuthorBio()` et choisit l'affichage.

→

3. BooksService + API REST
Envoie `GET .../authors/{id}` . La réponse HAL devient un `AuthorDetails` , puis un GET suit le lien `bio` .

Répartition du travail

Hjiyej Andaloussi Elghali — `Logique · commit 1`

- Création du dépôt GitHub et `.gitignore`
- Modèle `author-details.ts` : format HAL, conversion, vérification de l'id
- `BooksService` : `getAuthor` et `getAuthorBio`
- Routes ; logique des deux composants (états, messages, requêtes annulées)
- 24 tests unitaires et script `seed-data.sh`

Brayan Adou — `Interface · commit 2`

- Option Authors dans la barre de navigation, page active mise en évidence
- Templates HTML des deux composants : formulaire, fiche de l'auteur, messages
- Feuilles de style aux couleurs de l'application
- 8 tests d'affichage
- README, captures et rapport

Deux corrections faites pendant l'intégration

1. **L'ancien résultat restait affiché sous une erreur.** Après une recherche de l'id 99 puis de « abc », le message « Author not found » restait visible. Un id invalide ramène maintenant sur `/authors` (vérifié par un test).
2. **Le bouton Search devenait bordeaux après un clic.** Bootstrap 5.3 colore ses boutons avec ses propres variables CSS : le bouton les définit maintenant avec les couleurs de la barre de navigation.

02 Requête à l'API REST

Partie logique. Le service fait les requêtes HTTP. Les composants ne manipulent qu'un objet simple, `AuthorDetails`, obtenu à partir de la réponse HAL de l'API.

```ts
// books/service/books.service.ts
const Url = 'http://localhost:8080/books-api/';

// 1. L'auteur demandé (réponse HAL convertie)
public getAuthor(id: string | number)
    : Observable<AuthorDetails> {
  return this.http
    .get<AuthorResource>(Url + 'authors/' + id)
    .pipe(map(r => toAuthorDetails(r)));
}

// 2. Sa biographie : on suit le lien « bio »
public getAuthorBio(author: AuthorDetails)
    : Observable<string> {
  const bioUrl = author.bioUrl
    ?? Url + 'authors/' + author.id + '/bio';
  return this.http.get<BioResource>(bioUrl)
    .pipe(map(bio => bio.biodata ?? ''));
}
```

```ts
// authors/authors.component.ts (/authors)
onSubmit(): void {
  this.submitted = true;
  // " 007 " donne "7" ; vide, 0, 2.5, abc : null
  const id = normalizeAuthorId(this.authorId.value);
  if (id === null) {
    this.authorId.markAsTouched(); // message
    // + retour sur /authors s'il y avait
    //   le résultat d'une autre recherche
    return;
  }
  this.router.navigate(['./', id],
                       {relativeTo: this.route});
}
```

```ts
// authors/author/author.component.ts
// page /authors/:id
ngOnInit(): void {
  // émet à chaque nouvel id dans l'adresse
  this.paramSubscription = this.route.paramMap
    .subscribe(p =>
      this.loadAuthor(p.get('id') ?? ''));
}

loadAuthor(rawId: string): void {
  this.cancelRequests(); // recherche d'avant
  const id = normalizeAuthorId(rawId);
  if (id === null) {
    this.status = 'invalid-id'; // pas de requête
    return;
  }
  this.status = 'loading';
  this.authorSubscription = this.booksService
    .getAuthor(id).subscribe({
      next: author => {
        this.author = author;
        this.status = 'found';
        this.loadBio(author); // 2e requête
      },
      error: (e: HttpErrorResponse) =>
        this.showError(e)
    });
}

private showError(e: HttpErrorResponse) {
  if (e.status === 404) {
    this.status = 'not-found';
  } else if (e.status === 0) {
    this.status = 'unreachable'; // serveur arrêté
  } else { this.status = 'error'; }
}
```

États de la page d'un auteur

ÉTAT	QUAND	CE QUI EST AFFICHÉ
`loading`	Requête en cours	« Loading author 1… » avec un indicateur
`found`	Réponse 200	Fiche de l'auteur ; la biographie arrive avec une 2e requête
`not-found`	Réponse 404	« Sorry, no author was found with id 99. »
`invalid-id`	Adresse comme `/authors/abc`	« "abc" is not a valid author id. Please enter a whole number greater than 0. »
`unreachable`	Aucune réponse	« Unable to reach the Books API. Make sure the server is running on http://localhost:8080. »
`error`	Autre code (500…)	« The server could not return author 1 (error 500). Please try again. »

02 Réponse HAL et fichiers

L'API applique HATEOAS : elle renvoie peu de données, avec des liens pour aller plus loin. Le client s'en sert pour charger la biographie et pour ouvrir chaque livre.

`GET http://localhost:8080/books-api/authors/1`

```
{
  "id": 1,
  "firstName": "Koredo",
  "lastName": "Tablanga",
  "books": [
    { "title": "SpringBoot JPA and REST",
      "_links": { "self": {
        "href": ".../books-api/books/1" } } },
    { "title": "Angular for Beginners",
      "_links": { "self": {
        "href": ".../books-api/books/2" } } }
  ],
  "_links": {
    "self": {
      "href": ".../books-api/authors/1" },
    "bio": {
      "href": ".../books-api/authors/1/bio" }
  }
}
```

`authors/model/author-details.ts`

```
export function toAuthorDetails(
    r: AuthorResource): AuthorDetails {
  return {
    id: r.id,
    firstName: r.firstName,
    lastName: r.lastName,
    books: (r.books ?? []).map(book => ({
      // ".../books-api/books/7" donne 7
      id: idFromHref(book._links?.self?.href),
      title: book.title
    })),
    bioUrl: r._links?.bio?.href ?? null
  };
}

// résultat pour l'auteur 1
{ id: 1, firstName: 'Koredo',
  lastName: 'Tablanga',
  books: [{ id: 1, title: 'SpringBoot ...' },
          { id: 2, title: 'Angular ...' }],
  bioUrl: '.../books-api/authors/1/bio' }
```

Livres

Seul le titre est fourni. L'id lu à la fin du lien `self` sert à créer un lien vers `/books/1`, la page Books du code de départ.

Biographie

Absente de la réponse : le client suit le lien `bio`, qui renvoie `{ "id": 1, "biodata": "..." }`. Vide ou en erreur, l'auteur reste affiché.

Id introuvable

Le contrôleur renvoie `ResponseEntity.notFound()` : un code 404 sans contenu, que `HttpClient` transmet comme une erreur.

Fichiers principaux

FICHIER (CLIENT/SRC/APP)	RÔLE
`app.routes.ts`	Routes `/authors` et `/authors/:id`, sur le modèle de `/books` et `/books/:id`
`authors/model/author-details.ts`	Format HAL de l'API, `AuthorDetails`, conversion et vérification de l'id
`books/service/books.service.ts`	`getAuthor(id)` et `getAuthorBio(auteur)`, ajoutés aux méthodes existantes
`authors/authors.component.*`	Formulaire, message sous le champ, navigation, champ rempli d'après l'adresse
`authors/author/author.component.*`	Requêtes, six états, fiche de l'auteur, livres et biographie
`app.component.*`	Option Authors dans la barre de navigation
`scripts/seed-data.sh` (racine)	Remplit la base vide par l'API : 5 livres, 4 auteurs et leurs biographies

02 Tests et partage en deux commits

Chaque partie a ses tests. Les requêtes HTTP sont simulées avec `HttpTestingController`, et le vrai routeur Angular est utilisé (`RouterTestingHarness`).

Tests unitaires (Karma et Jasmine)

FICHIER DE TEST	CE QUI EST VÉRIFIÉ	DÉPART	LOGIQUE	INTERFACE
`author-details.spec.ts`	Lecture de l'id dans un lien, vérification de l'id, conversion HAL		7	
`books.service.spec.ts`	URL et méthode GET, conversion, erreur 404, lien `bio` suivi	1	4	
`authors.component.spec.ts`	Champ vide ou invalide, navigation vers `/authors/7`, ancien résultat retiré ; formulaire affiché		6	2
`author.component.spec.ts`	Les six états, biographie vide ou en erreur, requête annulée ; fiche et messages affichés		7	5
`app.component.spec.ts`	Option Authors placée après Books, lien vers `/authors`	2		1
Autres fichiers	Tests du code de départ (Home, About, Books, Book, Admin, Login, pipe, garde…)	11		
Total : 46		**14**	**24**	**8**

Les **46 tests** réussissent : 14 du code de départ, 24 de la partie logique (Elghali) et 8 de la partie interface (Brayan).

```
cd client
npx ng test --watch=false    # 46 tests : modèle, service, formulaire, fiche, menu
npx ng build                 # version de production dans dist/book-store/
```

Les deux avertissements de `ng build` (budget de 512 kB dépassé, 2 règles de Bootstrap ignorées) existent déjà dans le code de départ : ils viennent de la feuille de style Bootstrap.

Deux commits sur le même dépôt

Commit 1 · Hjiyej Andaloussi Elghali

Crée le dépôt avec le code de départ et la partie logique. L'application compile et fonctionne déjà : les pages `/authors` et `/authors/:id` ont des templates simples. **38 tests** réussissent.

Commit 2 · Brayan Adou

Récupère le dépôt (`git pull`), remplace 11 fichiers (menu, templates, CSS, tests d'affichage, README) et ajoute `docs/` (captures et rapport). **46 tests** réussissent.

Vérifié avant la remise. Les deux commits ont été rejoués dans un dépôt Git de test. Après chaque commit, le dépôt a été cloné à neuf, compilé (`ng build`) et testé (`ng test`). Les scripts `gradlew` et `seed-data.sh` restent exécutables.

03 Affichage et mise en forme

Partie interface : option du menu, templates HTML et feuilles de style des deux composants, dans le style de l'application de départ.

Choix de mise en forme (HTML et CSS)

La page Authors reprend Bootstrap 5, déjà chargé par `angular.json`, et les couleurs de l'application : le bleu de la barre de navigation, le rouge du survol et le bleu des titres de la page d'accueil. Chaque composant n'ajoute qu'une petite feuille de style (moins de 2 kB, la limite fixée dans `angular.json`).

Menu, Search	Page active	Barre active	Survol	Titres, fiche	Introuvable
#0000b3	#00007a	#ffc107	#e60000	#3465a4	#fff3cd

```
<!-- app.component.html : après Books -->
<a [routerLink]="['/books']"
   routerLinkActive="active"> Books </a>
<a [routerLink]="['/authors']"
   routerLinkActive="active"> Authors </a>

<!-- authors.component.html -->
<input id="authorId" class="form-control"
  formControlName="authorId"
  [class.is-invalid]="showError">
<button type="submit"
  class="btn btn-search">Search</button>
@if (showError) {
  <div class="invalid-feedback">
    {{ validationMessage }}</div>
}
<router-outlet></router-outlet>
```

```
<!-- author.component.html : un bloc par état -->
@switch (status) {
  @case ('loading') {
    <div role="status"> indicateur + message </div> }
  @case ('found') {
    <article class="card author-card">
      Author #{{ author.id }}, {{ fullName }}
      @for (book of author.books; track $index) {
        <a class="book-link"
           [routerLink]="['/books', book.id]">
          {{ book.title }}</a>
      }
      {{ biography }}
    </article> }
  @case ('not-found') {
    <div class="alert alert-warning">
      Author not found. {{ message }}</div> }
  @default {
    <div class="alert alert-danger"> ... </div> }
}
```

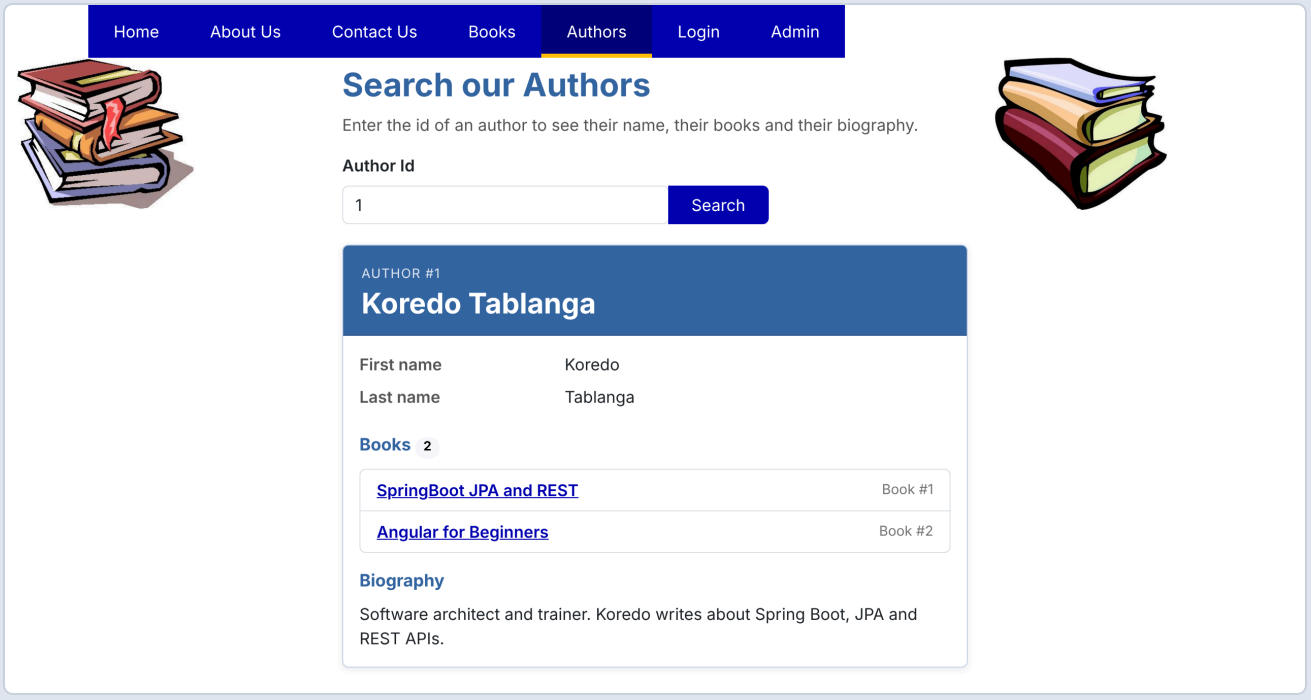
Éléments de l'interface

ÉLÉMENT	STYLE APPLIQUÉ
Menu	Option Authors après Books ; page ouverte en bleu foncé avec une barre ambre (`routerLinkActive`)
Formulaire	Titre bleu, phrase d'aide, champ et bouton Search groupés (`input-group`), exemple « e.g. 1 » dans le champ
Validation	Bord rouge, icône et message sous le champ (`is-invalid`) ; rien n'est envoyé tant que l'id est invalide
Fiche de l'auteur	Carte à en-tête bleu (« Author #1 », prénom et nom), puis prénom et nom séparés
Livres	Nombre de livres ; liste où chaque titre est un lien vers `/books/:id`, avec l'id du livre à droite
Biographie	Texte, ou « No biography available. » en italique, ou un message rouge si elle n'a pas pu être chargée
Messages	Indicateur de chargement ; alerte jaune (introuvable) ou rouge (id invalide, serveur)
Accessibilité	`role="status"`, `role="alert"`, `aria-invalid` ; contour visible au clavier

04 Captures d'écran

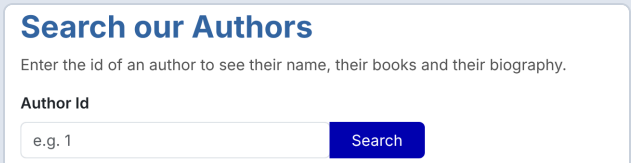
Client lancé avec `npm start` (`ng serve`) et ouvert à l'adresse `http://localhost:4200/`, après l'ajout des données d'exemple avec `scripts/seed-data.sh`.

`localhost:4200/authors/1`



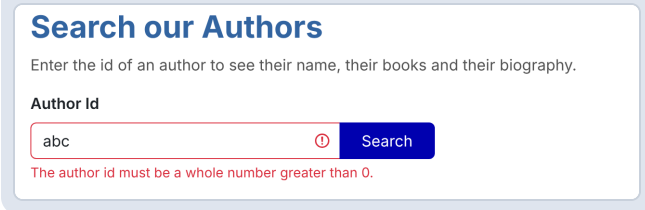
Auteur trouvé. Clic sur **Authors**, id 1, puis **Search** : l'adresse devient `/authors/1`. La fiche montre Koredo Tablanga, ses deux livres (liens vers leur page) et sa biographie, chargée par le lien `bio`.

`localhost:4200/authors`



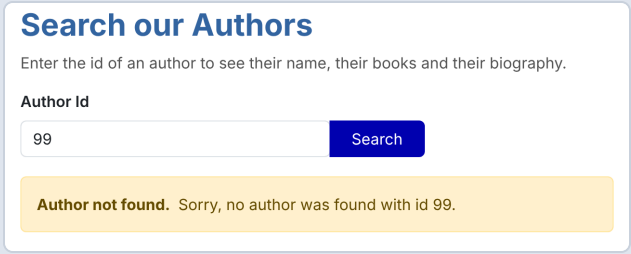
Option Authors. Le menu affiche le formulaire dans la zone Contenu ; l'option ouverte est mise en évidence dans la barre.

`localhost:4200/authors`



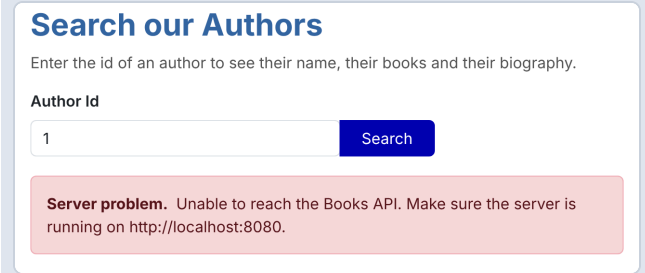
Id invalide. « abc » est refusé avant tout envoi : le champ devient rouge et le message apparaît dessous.

`localhost:4200/authors/99`



Auteur introuvable. L'API répond 404 pour l'id 99 : un message l'indique au lieu d'une fiche vide.

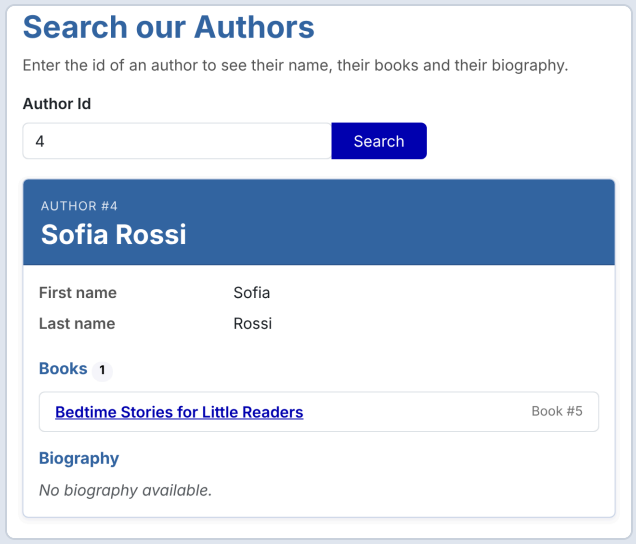
`localhost:4200/authors/1`



Serveur arrêté. Sans réponse de l'API, la page explique le problème.

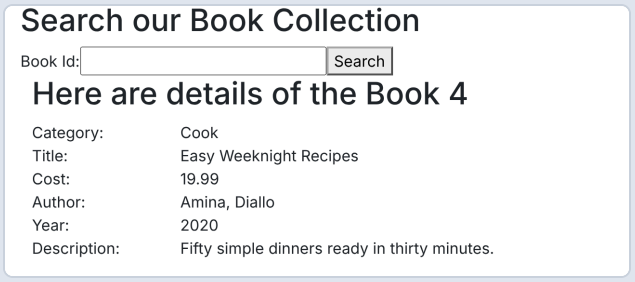
04 Captures d'écran (suite)

localhost:4200/authors/4



Sans biographie. Sofia Rossi (id 4) a un livre, mais sa biographie est vide.

localhost:4200/books/4



Lien vers un livre. Dans la fiche d'Amina Diallo (id 2), un clic sur « Easy Weeknight Recipes » ouvre la page Books de ce livre.

Ce que montrent les captures

- L'option Authors ouvre un formulaire pour entrer un id.
- L'envoi fait une requête et affiche l'auteur trouvé.
- Un message apparaît si l'auteur n'existe pas, si l'id est invalide ou si le serveur ne répond pas.
- Les liens HATEOAS mènent à la biographie et aux livres.

Parcours de test conseillé

Après `bash scripts/seed-data.sh` sur une base neuve, menu **Authors** :

ID ENTRÉ	RÉSULTAT ATTENDU
`1`	Koredo Tablanga, 2 livres (SpringBoot JPA and REST, Angular for Beginners) et sa biographie
`2`	Amina Diallo, 2 livres (Angular for Beginners, Easy Weeknight Recipes) et sa biographie
`3`	Marc Tremblay, 1 livre (Kotlin in Practice) et sa biographie
`4`	Sofia Rossi, 1 livre, « No biography available. »
`99`	« Author not found. Sorry, no author was found with id 99. »
`abc`, `0`, vide	Message sous le champ, aucune requête envoyée
`1`, serveur arrêté	« Server problem. Unable to reach the Books API… »

05 Installer et exécuter le projet

Deux dossiers dans le dépôt : `server/` (Spring Boot 3.1, Kotlin, Gradle) et `client/` (Angular 18). Trois terminaux sont utiles : serveur, données, client.

Prérequis

- **Docker** pour MySQL ; **Java 17** (le wrapper Gradle 8.0 du projet ne démarre pas avec Java 21, vérifier avec `java -version`).
- **Node.js** 18.19+, 20.11+ ou 22+ (`node -v`) ; npm est fourni avec Node.js. **Git** (Git Bash sous Windows pour le script de données).

Étapes

1 Cloner le dépôt

```
git clone https://github.com/ralihji/Lab-7.git
cd Lab-7
```

2 Démarrer MySQL (port 6033, base booksDb, root / root)

```
docker run -d -p 6033:3306 --name=docker-mysql --env="MYSQL_ROOT_PASSWORD=root" \
  --env="MYSQL_PASSWORD=root" --env="MYSQL_DATABASE=booksDb" mysql
```

Les fois suivantes : `docker start docker-mysql`. Au premier démarrage, attendre une vingtaine de secondes.

3 Démarrer le serveur Spring Boot

```
cd server
./gradlew bootRun          # Windows : gradlew.bat bootRun
```

API : `http://localhost:8080/books-api` ; Swagger UI : `http://localhost:8080/swagger-ui.html`.

4 Ajouter les données d'exemple (à la racine du dépôt)

```
bash scripts/seed-data.sh
```

La base est vide au départ. Le script ajoute 5 livres et 4 auteurs, puis affiche leurs ids (sur une base neuve : 1 Koredo Tablanga, 2 Amina Diallo, 3 Marc Tremblay, 4 Sofia Rossi).

5 Démarrer le client Angular

```
cd client
npm install
npm start                  # ou : npx ng serve
```

6 Ouvrir l'application

à l'adresse `http://localhost:4200/`, cliquer sur **Authors**, entrer **1** puis **Search**. Un id inexistant (99) affiche « Author not found ». Arrêter avec `Ctrl + C` dans chaque terminal, puis `docker stop docker-mysql`.

> **Ouvrir le client avec `localhost`, pas `127.0.0.1`.** Le serveur n'accepte que les requêtes venant de `http://localhost:4200` (`@CrossOrigin` dans `ApiController`). Sinon, la page Authors affiche « Unable to reach the Books API ».